HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

FUNDAMENTALS OF OPTIMIZATION

Metaheuristic methods

ONE LOVE. ONE FUTURE.

CONTENT

- Introduction to Metaheuristics
- Iterated Local Search
- Simulated Annealing
- Guided Local Search
- Variable Neighborhood Search
- Tabu Search

Metaheuristics

- Goal of heuristics is to reach high-quality local minima quickly.
- Metaheuristics aim at escaping these local minima and a directing the search toward global optimality.
  - Iterated Local Search
  - Simulated Annealing
  - Guided Local Search
  - Variable Neighborhood Search
  - Tabu Search

Iterated Local Search

- It is a ubiquitous metaheuristics that iterates a specific local search from different starting points in order to sample various regions of the search space and to avoid returning a low-quality local minimum.

```
1.  function IteratedLocalSearch(f, N, L, S) {
2.      s := GenerateInitialSolution();
3.      s* := s;
4.      for k := 1 to MaxSearches do
5.          s := LocalSearch(f, N, L, S, s);
6.          if f(s) < f(s*) then
7.              s* := s;
8.          s := GenerateNewSolution(s);
9.      return s*;
10. }
```

Simulated Annealing (1)

- Simulated annealing is a popular metaheuristic based on the Metropolis heuristic.
- Metropolis heuristic accepts a degrading move with probability

$$\exp\left(\frac{-(f(n)-f(s)}{t}\right)$$

- Where t is a parameter of the heuristic. Different values of t procedures different trade-offs between the quality of the solutions and the execution time.

Simulated Annealing (2)

- The key idea underlying simulated annealing is to iterate the Metropolis algorithms with a sequence of decreasing temperatures

$$t_0, t_1, \dots, t_i, \dots, (t_{k+1} \leq t_k\}$$

- The goal is to accept many moves initially in order to sample the search space widely (large values of t_k) to move progressively toward small values of t_k, thus covering toward random improvement and we hope a high-quality local minimum when $t_i \to 0$

Simulated Annealing (3)

```
1.  function SimulatedAnnealing(f, N) {
2.      s := generateInitialSolution();
3.      t_1 := initTemperature(s);
4.      s* := s;
5.      for k := 1 to MaxSearches do
6.          s := LocalSearch(f, N, L-All, S-Metropolis[t_k], s);
7.          if f(s) < f(s*) then
8.              s* := s;
9.          t_{k+1} := updateTemperature(s, t_k);
10.     return s*;
11. }
```

Guided Local Search (1)

- Guided Local Search is based on the recognition that a local optima s for the objective function f might not be locally optimal with respect to another function f'. Using f' instead of f would thus drive the search away from s.
- The key idea behind guided local search is to use a sequence of the objective functions $f_0, f_1, \ldots, f_i$ to drive the search away from local optima and to explore the search space more extensively.

Guided Local Search (2)

```
1.  function GuidedLocalSearch(f, N, L, S) {
2.      s := generateInitialSolution();
3.      f_1 := f;
4.      s* := s;
5.      for k := 1 to MaxSearches do
6.          s := LocalSearch(f_k, N, L, S, s);
7.          if f(s) < f(s*) then
8.              s* := s;
9.          f_{k+1} := updateObjective(s, f_k);
10.     return s*;
11. }
```

Variable Neighborhood Search (1)

- It works with a collection of neighborhoods $N_1, N_2, \dots, N_i$ to diversify the current solution.
- The intuition is that the neighborhoods $N_1, N_2, \dots, N_i$ are increasing in size, providing more opportunities for significant diversifications over time.

Variable Neighborhood Search (2)

```
1.  function VariableNeighborhoodSearch(f, N, L, S) {
2.      s := GenerateInitialSolution();
3.      s* := s;
4.      k := 1;
5.      while k ≤ MaxShaking do
6.          s := SELECT n ∈ N_k(s);
7.          s+ := LocalSearch(f, N, L, S, s);
8.          k := k + 1;
9.          if f(s+) < f(s*) then
10.             s := s+;
11.             s* := s;
12.             k := 1;
13.     return s*;
14. }
```

Tabu Search (1)

```
1.  function LocalSearch(f, N, L, S, s) {
2.      s* := s;
3.      for k := 1 to MaxTrials do
4.          if satisfiable(s) ∧ f(s) < f(s*) then
5.              s* := s;
6.          s := S(L(N(s), s), s);
7.      return s*;
8.  }
```

$$\tau = \langle s_0, s_1, \dots, s_k \rangle$$

Tabu search: Given a sequence $\langle s_0, s_1, \dots, s_k \rangle$, select s_{k+1} to be the best neighbor in $N(s_k)$ that has not yet been visited.

```
1.  function LocalSearch(f, N, L, S, s1) {
2.      s* := s1;
3.      τ := ⟨s⟩;
4.      for k := 1 to MaxTrials do
5.          if satisfiable(s) ∧ f(sk) < f(s*) then
6.              s* := sk;
7.          sk+1 := S(L(N(sk), τ), τ);
8.          τ := τ :: sk+1;
9.      return s*;
10. }
```


14

Tabu Search (2)

- Tabu search can be viewed as the combination of a greedy strategy with a definition of legal moves ensuring that a solution is never visited twice.
  - Allow the LS to select moves degrading the quality of the current solution
  - Greedy nature of the Tabu search ensures that the objective function does not degrade too much.

```
1.  function TabuSearch(f, N, s)
2.      return LocalSearch(f, N, L-NotTabu, S-Best);

where

1.  function L-NotTabu(N, τ)
2.      return { n ∈ N | n ∉ τ };
```

Tabu Search (3)

- Short-Term Memory
  - Difficulty to keep track of all visited solutions: computation time, memory
  - Short-term memory may not prevent the search from revisiting solution entirely, so that it is typically combined with other techniques
  - Transition abstraction that stores the transitions, not the states, since moves only involve a few variables in general.

Tabu Search (4)

- Aspiration:
  - Since Tabu Search stores sequence of abstractions and not sequences of solutions, it may forbid transitions $s_1 \rightarrow s_2$, in which s_2 has not been visited before.
  - Some of these transitions may be quite desirable, for instance when s_2 would be the best solution found so far $(f(s_2) \leq f(s^*))$
  - Aspiration feature can help the search overcome this limitation when the Tabu status may be overridden. The simplest and most widely used aspiration criterion overrides the tabu status of those moves that improve the best solution found so far.

Tabu Search (5)

- Long-Term Memory
  - Tabu list abstractions a small suffix of the solution sequence and cannot capture long-term information. As a consequence, it cannot prevent the search from taking long walks whose solutions have low-quality objective values or spending too much time in the same region, leaving other parts of the search space unexplored.
  - Many tabu search maintain a long-term memory structures to intensify and diversify the search.

Exercise

- **Problem 1**: Propose and implement a Tabu Search or other Metaheuristics Search for N-Queen problem.

- **Problem 2**: Propose and implement a Tabu Search or other Metaheuristics Search for Travelling Salesman Problem.

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !